Latto-Latto Control Research Changelog

Last updated: 2026-05-22

This document records major changes to the Latto-latto model, reward design, and training setup.

Rule for future updates. Any major change to the model, reward, controller, training setup, or evaluation protocol should be added to this file.

2026-05-22

Change 1: Reward Updated to Penalize Vertical Drift

Motivation.

- The original sparse reward was able to produce bouncing behavior, but the pivot vertical position z could drift over time.
- This made the learned behavior less physically meaningful and less suitable for fair controller comparison.

Previous reward.

$$r_t = \begin{cases} 1, & \text{if a collision occurs and the collision side alternates,} \\ 0, & \text{otherwise.} \end{cases}$$

Updated reward.

$$r_t = r_t^{\text{collision}} - \alpha z_t^2$$

with

$$r_t^{\text{collision}} = \begin{cases} 1, & \text{if a collision occurs and the collision side alternates,} \\ 0, & \text{otherwise.} \end{cases}$$

where:

- z_t is the vertical position of the pivot.
- α is the vertical position penalty weight.

Current setting in code.

$$\alpha = 0.1$$

Implementation notes.

- The reward is implemented in `latto_latto_model.py`.
- The environment now exposes reward components through the `info` dictionary:
  - `sparse_reward`
  - `z_position_penalty`

Observed outcome.

- PPO training completed successfully with the modified reward.
- The learned policy strongly reduced vertical drift.
- In quick evaluation, the policy also collapsed to a near-stationary solution with zero alternating collisions.

Interpretation.

- The drift penalty successfully regularized the base motion.
- The penalty weight $\alpha = 0.1$ appears too strong relative to the sparse collision reward.
- A smaller penalty weight, such as 0.01 or 0.02, should be tested next.

Change 2: Separate PPO Checkpoint for Reward-Penalty Experiment

Motivation. Preserve the original PPO checkpoint while training a new policy with the modified reward.

Change. Training output path changed from `ppo_latto` to `ppo_latto_z_penalty`.

Result. The reward-penalty experiment can now be evaluated independently from the original sparse-reward PPO model.

Change 3: Testing Script Updated to Load the New Checkpoint

Motivation. Keep the default evaluation script aligned with the most recent model variant.

Change. `testing_latto.py` now loads `ppo_latto_z_penalty`.

Note. If multiple controller or reward variants are added later, this script should eventually accept a model path as an argument instead of hardcoding one checkpoint.

Change 4: Visualization Workflow Improved

Motivation. The rendering logic was previously tied to repeated global `matplotlib` calls, which was less stable for repeated runs.

Change.

- Rendering now uses persistent `matplotlib` figure and axes handles.
- The environment now includes a `close()` method to release plotting resources cleanly.
- `testing_latto.py` now handles interactive and non-interactive backends more safely.

Research relevance. This is not a dynamics change, but it improves repeatability of visual inspection during controller evaluation.

Change 5: Testing Script Now Plots Pose Trajectories

Motivation.

- The previous testing workflow only saved the control input history.
- For controller comparison, it is also important to inspect how the pivot position and pendulum angle evolve over time.

Change.

- `testing_latto.py` now records and plots:
  - pivot vertical position $z(t)$
  - pendulum angle $\theta(t)$
- The script now saves an additional figure, `pose.png`.

Research relevance. This improves the evaluation workflow by making drift, oscillation amplitude, and near-stationary collapse easier to detect visually.

Change 6: Parameterized Alpha Sweep for Reward Comparison

Motivation.

- The first drift-penalty experiment with $\alpha = 0.1$ improved vertical centering, but it suppressed alternating collisions too strongly.
- A fair next step is to compare several smaller α values under the same PPO setup.

Change.

- `latto_latto_model.py` now accepts the vertical position penalty weight as a constructor argument.
- Added `alpha_sweep_experiment.py` to train and evaluate PPO models for:
  - $\alpha = 0.01$
  - $\alpha = 0.02$
  - $\alpha = 0.05$
- The sweep script saves:
  - per-alpha PPO checkpoints
  - `alpha_sweep_results.json`
  - `alpha_pose_comparison.png`

Research relevance. This turns the reward-tuning question into a reproducible experiment and directly supports comparison of $z(t)$ and $\theta(t)$ across reward weights.

Change 7: Collision Dynamics Updated from Ideal Reflection to Inelastic Impact

Motivation.

- The previous collision model used a perfect sign flip of angular velocity.
- That assumption preserved too much energy at impact and made the hybrid dynamics less realistic.
- For a stronger paper, the collision event should include explicit impact loss.

Previous collision reset.

$$\dot{\theta}^+ = -\dot{\theta}^-$$

Updated collision reset.

$$\dot{\theta}^+ = -e\dot{\theta}^-$$

where:

- $\dot{\theta}^-$ is the angular velocity immediately before impact.
- $\dot{\theta}^+$ is the angular velocity immediately after impact.
- $e \in (0, 1]$ is the coefficient of restitution.

Current setting in code.

$$e = 0.9$$

Impact energy loss proxy.

$$\Delta E_{\text{impact}} = \frac{1}{2}\left((\dot{\theta}^-)^2 - (\dot{\theta}^+)^2\right)$$

Implementation notes.

- `latto_latto_model.py` now accepts `collision_restitution` as a constructor argument.
- The environment now reports additional collision diagnostics through `info`:
  - `collision_flag`
  - `pre_collision_theta_dot`
  - `post_collision_theta_dot`
  - `collision_energy_loss`

Research relevance. This makes the environment a more meaningful hybrid impact-control benchmark. It also opens a stronger next experiment: compare controller robustness across different restitution values.

Change 8: Alpha Sweep Repeated on the Inelastic-Collision Model

Motivation.

- After introducing inelastic impacts with restitution $e = 0.9$, the previous alpha-sweep results no longer reflected the current model.
- The reward comparison needed to be repeated on the updated hybrid dynamics.

Change.

- `alpha_sweep_experiment.py` was updated to run the sweep on the current model with:
  - $e = 0.9$
  - $\alpha = 0.01$
  - $\alpha = 0.02$
  - $\alpha = 0.05$
- Separate artifacts are now saved for the inelastic sweep:
  - `ppo_latto_inelastic_alpha_0p010.zip`
  - `ppo_latto_inelastic_alpha_0p020.zip`
  - `ppo_latto_inelastic_alpha_0p050.zip`
  - `alpha_sweep_inelastic_results.json`
  - `alpha_pose_inelastic_comparison.png`

Observed outcome.

- $\alpha = 0.01$: `max |z| = 0.0250`, `mean |z| = 0.0092`, alternating collisions = `0`
- $\alpha = 0.02$: `max |z| = 0.0409`, `mean |z| = 0.0256`, alternating collisions = `0`
- $\alpha = 0.05$: `max |z| = 0.0313`, `mean |z| = 0.0121`, alternating collisions = `0`

Interpretation.

- $\alpha = 0.01$ remained the best of the three on vertical centering for this rollout.
- All three policies still collapsed to non-bouncing behavior in deterministic evaluation.
- The inelastic collision model made the task more realistic, but it did not by itself recover the desired alternating-impact rhythm under the current reward design.

Research relevance. This is a useful negative result. It suggests that model fidelity alone is not enough; the reward or controller structure likely also needs to change to sustain rhythmic impacts under dissipative collisions.